

# Mesure de la durée de vie du $D^0$ avec LHCb

Raoul Henderson

June 2026

## Résumé

Dans ce projet, vous analyserez des données collectées en 2012 par le détecteur LHCb, l'un des quatre détecteurs majeurs au Large Hadron Collider (LHC). Vous observerez un pic correspondant à la particule  $D^0$  se désintégrant en un pion ( $\pi^-$ ) et un kaon ( $K^+$ ). Vous effectuerez une sélection pour observer ce pic plus clairement, puis un fit pour extraire la masse du  $D^0$  et séparer le signal et le bruit de fond. Vous effectuerez un deuxième fit pour extraire la durée de vie du  $D^0$ .

## 1 Introduction

Ce projet simule une partie d'une analyse en physique des particules telles qu'elles peuvent être au sein de l'expérience LHCb. Le détecteur LHCb est l'un des quatre détecteurs majeurs du LHC au CERN, à Genève. Le LHC accélère des protons et les fait entrer en collision. Celles-ci libèrent une grande énergie qui entraîne la création de nombreuses particules. Un quark  $c$  et un antiquark  $\bar{u}$  peuvent *hadroniser* pour former un  $D^0$ . Cette particule est instable, et se désintègre ensuite. De nombreuses désintégrations sont possibles, et celle qui sera utilisée lors de ce projet est celle en deux particules chargées,  $\pi^-$  ( $d\bar{u}$ ) et  $K^+$  ( $u\bar{s}$ ).

Cette désintégration n'a pas lieu immédiatement : le  $D^0$  va, en moyenne, voler quelques millimètres dans le détecteur avant de se désintégrer. Le temps de désintégration suit une décroissance exponentielle, de demi-vie  $\tau_{D^0}$ . Le but de ce projet sera de mesurer  $\tau_{D^0}$ .

Un détecteur sur le LHC est un ensemble d'instruments se trouvant autour d'un des points de collisions entre les protons. Son objectif est de reconstruire l'entière des particules stables produites par la collision (ici on considérera uniquement les particules chargées) qui l'ont traversé. Pour cela, leur trajectoire est reconstruite à l'aide d'une série de positions où des *trajectographes* (trackers) ont été traversés et de légères quantités d'énergie ont été déposées. Cette trajectoire est courbée par un puissant champ magnétique, ce qui permet d'en déduire la charge et la quantité de mouvement de la particule. D'autres sous-détecteurs peuvent être présents (calorimètres pour les particules neutres, chambres à muons, ...) avec des rôles spécifiques.

LHCb, détaillé en Figure 1, est particulièrement adapté pour cela pour plusieurs raisons. Premièrement, il couvre une région angulaire dans laquelle de nombreux quarks  $c$  sont produits. Aussi, l'un de ses sous-détecteurs, le *Vertex Locator* permet une excellente précision sur la distance de vol des  $D^0$  et donc sur leur durée de vie. Enfin, certains de ses sous-détecteurs (*RICH1* et *RICH2*) permettent l'identification des kaons et des pions avec une certaine probabilité. Ces variables d'identification sont ensuite combinées avec

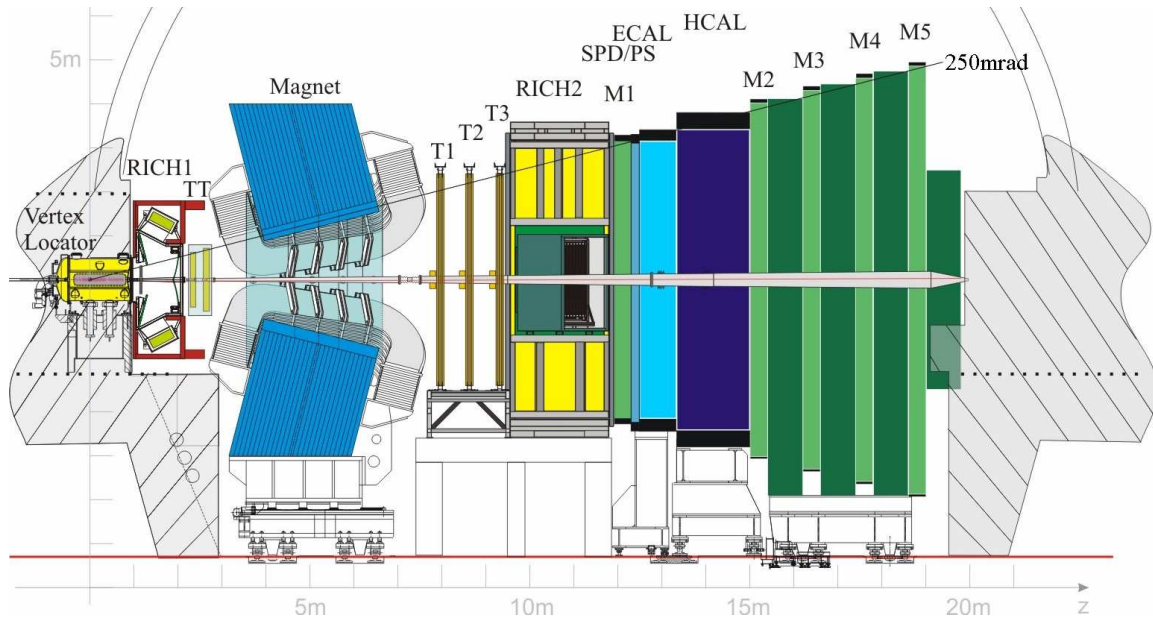


Figure 1: Le détecteur LHCb

d'autres informations sur la trace dans le détecteur par un réseau de neurone pour donner la probabilité que la trace corresponde à un pion ou un kaon. Ces variables sont appelées *ProbNN*.

Ces informations précises sont importantes, car des combinaisons aléatoires de traces laissées dans le détecteur peuvent ressembler aux traces que laisserait un  $D^0$  se désintégrant. Ces combinaisons s'appellent le bruit de fond combinatoire, et ne peuvent être différenciées du signal que grâce à la reconstruction la plus exacte possible de chaque événement. Malgré cela, une partie du bruit de fond restera toujours, même après une sélection poussée.

## 2 Outils

Les données fournies par LHCb à utiliser sont disponibles ici. Elles sont stockées sous le format `.root`. Il y a deux bibliothèques qui permettent communément d'explorer les fichiers `.root`, à savoir ROOT et uproot. ROOT est l'original, et peut être utilisé en C++ ou en python. Dans ce projet, cependant, uproot sera utilisé, car il est plus simple à intégrer aux bibliothèques standards de python (NumPy, pandas, matplotlib, ...). Les fits seront ensuite effectués à l'aide de zfit. Vous êtes encouragés à regarder ou effectuer le tutoriel de uproot. Le cas échéant, les sPlots peuvent obtenus avec la bibliothèque hepstats.

Certaines libraires (pandas, uproot, zfit) doivent être installées par vous-même. Pour ce faire, vous devez créer un environnement virtuel. Ouvrez un terminal et placez-vous dans le dossier de votre choix, où se trouve votre projet. Pour vous déplacer dans le terminal, utilisez

```
cd <dossier de votre choix>
```

et, pour remonter d'un dossier, utilisez

```
cd ..
```

Vous pouvez vous déplacer de plusieurs dossiers à la fois, avec

```
cd <dossier>/<sous-dossier>/<sous-sous-dossier>/...
```

Créez ensuite votre environnement avec

```
python -m venv <nom de l'environnement>
```

Puis vous devez activer cet environnement, avec

```
source <nom de l'environnement>/bin/activate
```

Vous pouvez ensuite installer les librairies nécessaires avec par exemple

```
pip install uproot
```

De même, installez jupyter notebook ou spyder selon votre préférence avec

```
pip install notebook
```

Vous pourrez alors lancer jupyter notebook ou spyder quand vous avez activé votre environnement, et celui-ci sera inclus. Pour désactiver votre environnement si nécessaire, utilisez

```
deactivate
```

Une fois votre environnement en place, les fois suivantes, déplacez vous simplement dans votre dossier de choix, activez votre environnement et lancez notebook ou spyder.

## 3 Projet

### 3.1 Préambule

Le fichier ROOT fourni par LHCb contient un seul objet d'intérêt, DecayTree. Celui-ci, contient tous les événements que vous allez analyser, sous la forme de branches. Chaque branche correspond à une caractéristique spécifique des événements étudiés, tel que la masse mesurée du candidat  $D^0$ . C'est une forme de grand tableau, dans lequel chaque valeur contenue correspond à la caractéristique en question pour un événement spécifique. Le DecayTree des données contient ici 13 branches. Celles-ci contiennent des vrais  $D^0$ , le signal, mais aussi des combinaisons aléatoires de  $\pi^-$  et  $K^+$  ayant passé les sélections précédentes, le bruit de fond. La distribution où la différence entre les deux est la plus apparente est la masse du  $D^0$ , où le bruit de fond est presque plat, alors que les vrais  $D^0$  forment un pic approximativement Gaussien (la masse devrait être toujours la même, mais la résolution du détecteur n'est pas parfaite).

1. Codez une fonction `charge_donnees` qui prend un fichier `.root` et une liste de branches, et renvoie un pandas dataframe correspondant.
2. Codez une fonction `dessinebranche` qui prend un dataframe et des noms de branches et sauvegarde un histogramme de chaque branche. Utilisez-la pour observer les différentes branches des données.
3. Quelles sont les branches :

- Donnant la masse du candidat  $D^0$ ?
- Donnant la durée de vie du candidat  $D^0$ ?
- Donnant la quantité de mouvement transverse ( $p_T$ ) de chaque particule ?
- Donnant la probabilité d'identification en kaon ou en pion pour chaque particule ?

La branche `D0_MINIP` correspond au paramètre d'impact du candidat  $D^0$  par rapport au point de collision, c'est-à-dire la distance minimale entre ce point et la trajectoire reconstruite du  $D^0$ . La branche `D0_MINIPCHI2` est similaire. Elle donne la différence de  $\chi^2$  entre la qualité de reconstruction de la trace selon qu'elle soit forcée de passer par le point de collision ou non. La branche `D0_DIRA_OWNPV` donne le cosinus de l'angle entre la quantité de mouvement du candidat  $D^0$  et la droite reliant son point de désintégration et le point de collision.

## 3.2 Sélection

En observant la distribution de la masse des candidats  $D^0$ , on peut observer une quantité importante de bruit de fond.

1. Après avoir observé les différentes branches, avec quelles branches est-il le plus simple d'éliminer une quantité importante de bruit de fond (à part la masse du  $D^0$ ), à l'aide de sélections simples à votre avis ? Modifiez `charge_données` pour qu'une sélection puisse être effectuée sur le dataframe renvoyé. Vérifiez si le ratio de signal par rapport au bruit de fond a bien augmenté.
2. Pour effectuer une sélection plus poussée, il faut un moyen de comparer les distributions du signal et du bruit de fond pour les différentes branches. Réfléchissez à une sélection qui permettrait de séparer les données en deux parties, l'une avec (majoritairement) le signal et l'autre avec (majoritairement) le bruit de fond.
3. Modifiez `dessine_branche` pour qu'elle puisse prendre plusieurs dataframes, dont les histogrammes seront sauvegardés sur le même dessin. Utilisez-la pour améliorer votre sélection.
4. Pouvez-vous expliquer pourquoi les distributions de `D0_MINIP` (ou `D0_MINIPCHI2`) et `D0_DIRA_OWNPV` sont différentes pour les vrais  $D^0$  et le bruit de fond ?

## 3.3 Extraction de la masse du $D^0$

Le but d'un fit est d'adapter un modèle choisi à des données, en obtenant la valeur optimale de paramètres de ce modèle. Pour ce faire, un modèle générique avec des paramètres libres est d'abord créé. Celui-ci est souvent décomposé en plusieurs sous-composants, chacun contribuant à une partie du nombre total d'évènement. Chaque modèle est représenté par une *Probability Density Function* (PDF).

Le fit cherchera ensuite à maximiser la *vraisemblance* (likelihood)  $\mathcal{L}(\theta, \mathbf{x})$ , construite à partir du modèle et son ensemble de paramètres  $\theta$  et des données  $\mathbf{x} = x_1, \dots, x_M$ . La vraisemblance totale est le produit de la vraisemblance de chaque point de donnée,  $\mathcal{L} = \prod_i \mathcal{L}(\theta, x_i)$ . Il est plus simple de manipuler des sommes que des produits, donc le fit maximise plutôt  $\ln(\mathcal{L}(\theta, \mathbf{x})) = \sum_i \ln(\mathcal{L}(\theta, x_i))$ . Enfin, comme il est plus simple de

minimiser que de maximiser en règle générale, le fit va en réalité minimiser  $-\ln(\mathcal{L}(\theta, \mathbf{x}))$ , la *log-vraisemblance négative* (negative log-likelihood).

Lorsque l'on souhaite obtenir le nombre d'évènements appelé *rendement* (yield), associé à chaque composant (ce qui se sera le cas dans ce projet), le fit est dit *étendu* (extended). Dans ce cas, du fait de la distribution Poissonienne du nombre d'évènements pour chaque composant, la log-likelihood devient

$$\ln(\mathcal{L}(\theta, \mathbf{x})) = \sum_i \ln\left(\sum_j N_j \cdot f_j(\theta, x_i)\right) - \sum_j N_j \quad (1)$$

avec la somme sur  $i$  correspondant à chaque évènement et la somme sur  $j$  correspondant à chaque composant, associé à son PDF  $f$ .

N'hésitez pas à effectuer le tutoriel indiqué sur zfit, que vous pourrez modifier pour effectuer les fits demandés.

1. Suggérez un modèle simple pour le signal et le bruit de fond. Vérifiez leur implémentation dans zfit.
2. Codez une fonction `fit_masse` qui fit la distribution de la masse des candidats  $D^0$ . Vous pouvez créer une fonction intermédiaire `fit_donnees` générique pour cela.
3. Quels sont les résultats de votre fit ? Comparez avec la masse connue du  $D^0$ , ici. Commentez.
4. Un bon moyen de vérifier la qualité de votre fit est de tracer les *pulls* de celui-ci. Il s'agit de tracer un histogramme de la distribution fittée, puis, dans chaque bin de l'histogramme, de calculer la différence entre la prédiction du modèle et les données, divisée par l'incertitude sur les données. Ensuite, tracer les pulls. Commentez.
5. Pour un nombre unique donnant une idée la qualité du fit, vous pouvez calculer le  $\chi^2$  de celui-ci. Pour cela, faites la somme du carré du pull dans chaque bin, puis divisez par le nombre de degrés de liberté (le nombre de bins – le nombre de paramètres flottants dans le fit). En général, plus le  $\chi^2$  est petit, meilleur est le fit, mais un  $\chi^2$  trop proche de 0 indique en général que le modèle a trop de paramètres (*overfitting*). Quel est le  $\chi^2$  de votre fit ?

### 3.4 Amélioration de la sélection

La sélection que vous avez effectuée en Sec. 3.2 peut être améliorée ou, au minimum, obtenue de manière plus systématique. D'une manière générale, la sélection cherche à maximiser le signal divisé par l'incertitude sur celui ci, ce qui s'appelle une *Figure de Mérite* (FOM).

1. Rappelez l'incertitude statistique sur un nombre d'évènements en fonction de ce nombre. Donnez donc l'expression de la Figure de Mérite choisie en fonction de S le signal et B le bruit de fond sous le signal.
2. Modifiez votre fonction `fit_masse` pour qu'elle renvoie S et B
3. Codez une fonction `FOM_variable` qui prend une variable et renvoie  $\frac{S}{\sqrt{S+B}}$  pour celle-ci et la sélection la maximisant. Tracez ensuite la FOM.

4. Utilisez-la pour optimiser votre sélection.
5. Essayez de maximiser la sélection, mais cette fois sans utiliser aucune sélection initiale sur les variables d'identification de particules. La sélection est-elle la même ou presque ? Pourquoi ? Quelles conséquences en tirez-vous ?

### 3.5 Extraction de la durée de vie du $D^0$

Pour rappel, le  $D^0$  se désintègre suivant une loi exponentielle décroissante, il est donc possible d'extraire  $\tau_{D^0}$  à partir de la distribution du temps de vol des candidats  $D^0$ .

1. Essayez de faire un fit de la durée de vie avec une simple exponentielle. Quelle est la qualité de ce fit ? Comparer  $\tau_{D^0}$  avec sa valeur connue, ici. Que pouvez vous en conclure ?
2. Comment obtenir un modèle de la distribution de la durée de vie pour le bruit de fond ?
3. Faites un fit avec ce modèle et l'exponentielle pour le signal. Commentez le résultat. La différence avec la valeur connue de  $\tau_{D^0}$  est-elle acceptable ?

### 3.6 Autre méthode avec $_sPlot$

On peut tenter d'utiliser une autre méthode pour le fit de la durée de vie du  $D^0$ . Plutôt que de modéliser le bruit de fond comme effectué en Sec. 3.5, on peut tenter de retirer ce bruit de fond des données. Les enjeux de ce type sont courants, une méthode a donc été développée pour cela : le  $_sPlot$ . Vous êtes invités à lire cette introduction pour plus de détails. La procédure de  $_sPlot$  associe un poids, appelé  $_sWeight$ , à chaque évènement. Celui-ci ne correspond pas exactement à la probabilité pour cet évènement d'être du signal ou du bruit de fond, mais l'ensemble des évènements, une fois leur poids "signal" appliqué, reproduit en moyenne la distribution du signal dans une autre variable, à une condition. Laquelle ?

Ces poids dépendent de la matrice de covariance  $V$ , définie pour les paramètres  $X_i$  et  $X_j$  par  $V_{ij} = \langle (X_i - \langle X_i \rangle)(X_j - \langle X_j \rangle) \rangle$ , où  $\langle X \rangle$  est la valeur attendue de  $X$ . Dans le cas d'un fit de la likelihood, cette matrice est l'inverse de la matrice Hessienne  $H$ , soit :

$$V_{ij} = H_{ij}^{-1} = \left( \frac{\partial(-\ln(\mathcal{L}))}{\partial X_i \partial X_j} \right)^{-1} \quad (2)$$

pour  $X_i, X_j$  prenant les valeurs maximisant  $\mathcal{L}$ . Enfin, le poids de signal d'un évènement de masse  $m$  dans notre contexte, avec  $S$  le signal et  $B$  le bruit de fond, est :

$$_sP_S(m) = \frac{V_{SS}f_S(m) + V_{SB}f_B(m)}{N_S f_S(m) + N_B f_B(m)} \quad (3)$$

avec  $f_{S(B)}$  le PDF modélisant le signal (bruit de fond). Vous pouvez utiliser la librairie `hepstats` pour obtenir les  $_sWeights$ .

1. Obtenez les  $_sWeight$  à partir du fit de la masse du  $D^0$ .

2. Tracer l'histogramme de la durée de vie du  $D^0$  pour le signal seulement, et pour le bruit de fond seulement. Prenez l'erreur associée à chaque bin. Quelle est-elle ? Pour le bruit de fond, comparez avec la distribution que vous avez déjà obtenue.
3. Modifiez votre fonction pour fitter la durée de vie pour qu'elle puisse prendre des poids pour chaque évènement.
4. Essayez de fitter cette distribution avec le signal uniquement en retirant le modèle du bruit de fond. Commentez le résultat.

### 3.7 Pour aller plus loin

- Quels autres modèles de pics ou bruits de fond plus précis pourriez-vous utiliser pour la masse ? Comparer la qualité de votre fit avec ces nouveaux modèles.
- Pouvez-vous extraire les poids du `Plot` sans utiliser une librairie existante ?
- Avez-vous une autre idée pour créer un histogramme de  $\tau_{D^0}$  contenant uniquement le signal à partir de la distribution de masse du  $D^0$ ?

N'hésitez pas si vous avez d'autres suggestions !